



16K+

Охват за 30 дней

АльфаСтрахование

4,79 Оценка работодателя

109,12 Рейтинг

222 Подписчики

[Подписаться](#)

nosbuka

19 часов назад

Трейсы в Spring Boot 3 с использованием Zipkin и Kafka в качестве транспорта. Часть 2

[🔗 Средний](#) [🕒 21 мин](#) [👁 5.3K](#)

Блог компании АльфаСтрахование

[Тutorial](#)

Трейсы в Spring Boot 3 с использованием Zipkin и Kafka в качестве транспорта. Часть 2

Предисловие

В [предыдущей статье](#) мы создали автоконфигурацию для того, чтобы включать трейсы в приложениях на базе Spring Boot. Как ранее было указано, она не позволяет собрать воедино цепочку вызовов из трейсов нескольких сервисов. В текущей статье мы доработаем стартер, добавив эту возможность.

Изменения, не относящиеся к решению проблемы

С момента написания первой части статьи, в стартер были добавлены изменения: появились конфигурационные свойства `custom.tracing.sasl-mechanism` и `custom.tracing.security-protocol`, которые передаются в свойства продюсера. Это позволяет управлять механизмами аутентификации в kafka.

Кроме того, мы стали явно указывать тип трейсов, добавив `management.tracing.propagation.type` со значением `w3c`. Это решило проблему смешивания различных типов трейсов, что ранее приводило к разрывам в трассировке.

Конечная конфигурация в `application.yaml` стала выглядеть следующим образом:

```
custom:
  tracing:
    bootstrap-servers: broker-1:9092, broker-2:9092, broker-3:9092
    username: ${USERNAME}
```

```
password: ${PASSWORD}
sasl-mechanism: SCRAM-SHA-256
security-protocol: SASL_SSL
topic: trace-topic
management:
  tracing:
    enabled: true
    propagation:
      type: w3c
    sampling:
      probability: 1
```

Объяснить с  SourceCraft

Как мы интегрируемся

Для понимания контекста и потребности в создаваемых конфигурационных bean'ах, нужно обозначить технологии, с помощью которых мы интегрируем сервисы:

- [Spring Cloud Openfeign](#): создаем клиенты при синхронных REST-интеграциях;
- [Spring Kafka](#) при взаимодействии через Kafka;
- [Apache CXF](#): используем для SOAP-интеграций (альтернативная ссылка на гайд Baeldung: [A Guide to Apache CXF with Spring](#));
- [Spring Cloud Gateway](#): используем для маршрутизации запросов между сервисами;

Формат трейсов

Мы будем использовать формат `w3c`, который следует [стандарту W3C Trace Context](#).

Когда используется W3C Trace Context, интеграционное взаимодействие включает следующие заголовки:

- `traceparent`: Основной ID трассировки и ID операции;
- `tracestate`: Дополнительная информация для передачи через границы систем (опциональный).

Traceparent

Это основной заголовок, который несет в себе:

- Версию протокола;
- Trace ID (глобальный идентификатор трассировки);
- Parent ID (идентификатор текущего span);
- Flags (флаги управления трассировкой).

Формат заголовка traceparent:

```
{version}-{trace-id}-{parent-id}-{trace-flags}
```

Объяснить с 

Здесь:

- 00 — Версия формата `traceparent` (сейчас используется: 00 (версия 0));
- 69bd60bfbe6c8f2ebc4fd6787aa0a747 — Trace ID (уникальный идентификатор одной трассы (end-to-end запрос));
- bc4fd6787aa0a747 — Parent ID (идентификатор конкретного span в трассе). В новой системе будет создан новый span с новым parent-id, но старым trace-id;
- 01 — Флаги управления трассировкой. В [спецификации sampled-flag](#) описывается, что существуют различные сценарии их обработки. В рамках текущего решения используется Probability sampling, вероятность которого задается свойством `management.tracing.sampling.probability` в `application.yaml`:
 - 01 : трассировка регистрируется в системе распределенной трассировки;
 - 00 : распространение трассировки не прерывается, но регистрация останавливается. При этом флаг распространяется далее.

Подробное описание приведено в [спецификации заголовка traceparent](#).

Tracestate

Этот заголовок используется для передачи вендор-специфичных данных и добавления информации в цепочку запросов без изменения стандарта `traceparent`. Подробное описание можно найти в [спецификации заголовка tracestate](#).

Конфигурации

В [предыдущей статье](#) все конфигурации были вынесены в отдельный стартер, поэтому изменять конечные приложения нам не потребуется (кроме изменения версии самого стартера).

Здесь и далее все дополнения производятся в исходном коде стартера, но если вы конфигурируете непосредственно приложение, то можно просто добавить конфигурации в него.

OpenFeign

Для feign-клиентов необходимо добавить поддержку micrometer. Существует несколько способов это сделать в т.ч. автоматически (при выполнении определенных условий). Мы создали bean типа `MicrometerObservationCapability`. Описание подхода описано в [спецификации Spring Cloud OpenFeign](#).

Конфигурация:

```
import feign.micrometer.MicrometerObservationCapability;
import io.micrometer.observation.ObservationRegistry;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration(proxyBeanMethods = false)
public class FeignObservedConfiguration {

    @Bean
    public MicrometerObservationCapability micrometerObservationCapability(
```

```

        ObservationRegistry registry
    ) {
        return new MicrometerObservationCapability(registry);
    }
}

```

[Объяснить с 🧑🏻💻](#)

Зависимости

Для получения класса `feign.micrometer.MicrometerObservationCapability` подключаем зависимость:

```

<dependency>
    <groupId>io.github.openfeign</groupId>
    <artifactId>feign-micrometer</artifactId>
    <version>13.5</version>
</dependency>

```

[Объяснить с 🧑🏻💻](#)

Результат

Любой вызов сторонних (по отношению к нашему) сервисов сопровождается добавлением HTTP-заголовка `traceparent`.

Kafka

Для работы с заголовком `traceparent` в сообщениях kafka нам потребуется включить `observation` для компонентов:

- для `listener`, в случае, если приложение занимается получением сообщений;
- для `template`, в случае, если приложение занимается отправкой сообщений.

Включение может быть произведено через `configuration properties`, вот так:

```

spring:
  kafka:
    listener:
      observation-enabled: true
    template:
      observation-enabled: true

```

[Объяснить с 🧑🏻💻](#)

Чтобы не писать эти свойства в конфигурации каждого конечного приложения, мы используем `BeanPostProcessor`, исходный код которого приведен ниже:

```

import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.kafka.config.ConcurrentKafkaListenerContainerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.lang.NonNull;

```

```

@Slf4j
public class KafkaObservationEnablerPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessAfterInitialization(
        @NonNull Object bean,
        @NonNull String beanName
    ) throws BeansException {
        if (bean instanceof ConcurrentKafkaListenerContainerFactory<?, ?> factory) {
            factory.getContainerProperties().setObservationEnabled(true);
            log.debug("Observation enabled in consumer [beanName: {}, class: {}]", beanName, bean.getClass());
        } else if (bean instanceof KafkaTemplate<?, ?> kafkaTemplate) {
            kafkaTemplate.setObservationEnabled(true);
            log.debug("Observation enabled in producer [beanName: {}, class: {}]", beanName, bean.getClass());
        }
        return bean;
    }
}

```

Объяснить с 

Реализация применяется ко всем listener'ам и template'ам приложения, включая необходимые свойства.

Более подробно подход описан в статье Baeldung: [Micrometer Observation and Spring Kafka](#).

Результат

Сообщения, отправленные в kafka, содержат заголовок `traceparent`. При получении сообщения, контекст трейса восстанавливается, все последующие действия привязываются к нему.

SOAP-интеграции

Прокси для SOAP-интеграций мы создаем с помощью `JaxWsProxyFactoryBean`. Если не углубляться в детали, то обычно это выглядит следующим образом:

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import ru.alfastrah.api.tracing.observability.soap.ws.KnownSoapClient;

@Configuration
class TestSoapClientsConfiguration {

    public static final String TEST_URL = "test-url";

    @Bean
    public KnownSoapClient knownSoapClient() {
        JaxWsProxyFactoryBean jaxWsProxyFactoryBean = new JaxWsProxyFactoryBean();
        jaxWsProxyFactoryBean.setAddress(TEST_URL);
        return jaxWsProxyFactoryBean.create(KnownSoapClient.class);
    }
}

```

```
}  
}
```

[Объяснить с](#) 

Для добавления HTTP-заголовка `traceparent` мы создаем реализацию `AbstractOutDatabindingInterceptor`, которую должны передать в `factory` `JaxWsProxyFactoryBean`.

Исходный код `interceptor` может выглядеть следующим образом:

```
import io.micrometer.tracing.TraceContext;
import io.micrometer.tracing.Tracer;
import org.apache.cxf.interceptor.AbstractOutDatabindingInterceptor;
import org.apache.cxf.interceptor.Fault;
import org.apache.cxf.message.Message;
import org.apache.cxf.phase.Phase;
import ru.alfastrah.api.tracing.util.Constants;

import java.util.List;
import java.util.Map;
import java.util.TreeMap;

import static java.util.Objects.isNull;
import static java.util.Objects.nonNull;

public class SoapTraceParentOutDatabindingInterceptor extends AbstractOutDatabindingInterceptor {

    private final Tracer tracer;

    public SoapTraceParentOutDatabindingInterceptor(
        Tracer tracer
    ) {
        super(Phase.PREPARE_SEND);
        this.tracer = tracer;
    }

    private static Map<String, List<String>> fetchHeaders(Message message) {
        Map<String, List<String>> headers = (Map<String, List<String>>) message.getMessageProperties().getHeaders();
        if (isNull(headers)) {
            headers = new TreeMap<>();
        }
        return headers;
    }

    @Override
    public void handleMessage(Message message) throws Fault {
        final TraceContext traceContext = tracer.currentTraceContext().context();
        if (nonNull(traceContext)) {
            final Map<String, List<String>> headers = fetchHeaders(message);
            headers.put(
                Constants.HttpHeaders.TRACE_PARENT,
                Constants.Helpers.TO_TRACE_PARENT.apply(traceContext)
            );
            message.getMessageProperties().put(Message.PROTOCOL_HEADERS, headers);
        }
    }
}
```

```

    }
}
}

```

Объяснить с 

Здесь компонент получает текущий контекст из переданного в конструктор

`io.micrometer.tracing.Tracer`, после чего формирует заголовок и добавляет его к сообщению.

Заголовок формируется функцией `Constants.Helpers.TO_TRACE_PARENT` типа

`Function<TraceContext, List<String>>`:

```

public static final Function<TraceContext, List<String>> TO_TRACE_PARENT = traceContext -> {
    "%s-%s-%s-%s".formatted(
        "00", // https://www.w3.org/TR/trace-context/#version
        traceContext.traceId(), // https://www.w3.org/TR/trace-context/#trace-id
        traceContext.parentId(), // https://www.w3.org/TR/trace-context/#parent-id
        Boolean.TRUE.equals(traceContext.sampled()) ? "01" : "00" // https://www.w3.org/TR/trace-context/#sampled
    );
}

```

Объяснить с 

Немного выше я привел пример того, как создается прокси. Следует помнить, что его создание происходит в приложении, тогда как конфигурация происходит в компоненте стартера. Отсюда возникает важная деталь: нужно передать `interceptor` в `JaxWsProxyFactoryBean`, при этом оставить контроль над трейс-конфигурацией стартеру, не передавая его приложению и не заставляя разработчика об этом помнить.

Решением данной задачи для нас служит `@Aspect`, содержащий `@Pointcut` 'ы для методов, которые создают прокси. Сразу отмечу, что количество SOAP-сервисов невелико, а все пакеты, которые генерируются `jax`ом для них, также известны.

```

import lombok.RequiredArgsConstructor;
import org.apache.cxf.endpoint.Client;
import org.apache.cxf.frontend.ClientProxy;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Aspect
@RequiredArgsConstructor
public class SoapClientCreationAspect {

    private final SoapTraceParentOutDataabindingInterceptor soapTraceParentOutDataabindingInterceptor;

    @Pointcut("execution(ru.alfastrah.api..ws.* *(..))")
    public void matchesSoapClientType() {
    }

    @Pointcut("@within(org.springframework.context.annotation.Configuration) && @annotation(org.springframework.context.annotation.Bean)")
    public void matchesConfiguration() {
    }
}

```

```

public void beanInConfiguration() {
}

@Pointcut("beanInConfiguration() && matchesSoapClientType()")
public void soapClientBeanCreation() {
}

@AfterReturning(pointcut = "soapClientBeanCreation()", returning = "proxy")
public Object afterUnicusSoapClientBeanCreated(Object proxy) {
    final Client client = ClientProxy.getClient(proxy);
    client.getOutInterceptors().add(soapTraceParentOutDatabindingInterceptor);
    return proxy;
}
}

```

[Объяснить с 📄](#)

Конфигурация вышеуказанных компонентов для включения аспекта:

```

import io.micrometer.tracing.Tracer;
import org.apache.cxf.jaxws.JaxWsProxyFactoryBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnClass;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration(proxyBeanMethods = false)
@ConditionalOnClass(JaxWsProxyFactoryBean.class)
public class SoapObservationConfiguration {

    @Bean
    @ConditionalOnBean(Tracer.class)
    SoapTraceParentOutDatabindingInterceptor soapTraceParentOutDatabindingInterceptor(
        Tracer tracer
    ) {
        return new SoapTraceParentOutDatabindingInterceptor(tracer);
    }

    @Bean
    @ConditionalOnBean(SoapTraceParentOutDatabindingInterceptor.class)
    SoapClientCreationAspect soapTracingAspect(
        SoapTraceParentOutDatabindingInterceptor interceptor
    ) {
        return new SoapClientCreationAspect(interceptor);
    }
}

```

[Объяснить с 📄](#)

Если вы конфигурируете трейсы в приложении, то реализация `@Aspect` 'а для перехвата создаваемых прокси может оказаться излишеством. Достаточно просто передать `interceptor` в `JaxWsProxyFactoryBean`.

Зависимости

Классы `org.apache.cxf.frontend.ClientProxy` и `org.apache.cxf.jaxws.JaxWsProxyFactoryBean` можно найти, подключив зависимость:

```
<dependency>
  <groupId>org.apache.cxf</groupId>
  <artifactId>cxf-rt-frontend-jaxws</artifactId>
  <version>4.1.5</version>
  <scope>provided</scope>
</dependency>
```

[Объяснить с](#) 

Результат

Все прокси типов `ru.alfastrah.api..ws.*`, создаваемые приложением, перехватываются и обогащаются дополнительным `interceptor`'ом, который добавляет HTTP-заголовок `traceparent` к вызовам SOAP-сервисов.

Spring Cloud Gateway

Интеграции (REST и SOAP) в нашем случае осуществляются с применением Spring Cloud Gateway, который маршрутизирует запросы на микросервисы. Помимо маршрутизации, у него есть иные задачи (добавление заголовков, аутентификация и т.п.), поэтому его участие также нужно видеть в цепочке трейсов для наиболее полной картины потока данных.

Spring Cloud Gateway написан на реактивном стеке, для его конфигурации потребуется поместить `ObservationRegistry` в инстанс `ObservationThreadLocalAccessor` и включить автоматическое распространение контекста. Эта конфигурация относится не только к Gateway, но и к сервисам, написанным на реактивном стеке.

Исходный код `InitializingBean` с конфигурацией, включающей его:

```
import io.micrometer.context.ContextRegistry;
import io.micrometer.observation.ObservationRegistry;
import io.micrometer.observation.contextpropagation.ObservationThreadLocalAccessor;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnWebApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Hooks;

@Configuration(proxyBeanMethods = false)
@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.REACTIVE)
public class ReactorObservationConfiguration {

    @Bean
    public InitializingBean reactorObservationInitializer(
        ObservationRegistry observationRegistry
    ) {
        return () -> {
            // Link ThreadLocal Micrometer with Reactor Context
            final ObservationThreadLocalAccessor threadLocalAccessor = ObservationThreadLocalAccessor.create();
            threadLocalAccessor.setObservationRegistry(observationRegistry);
            ContextRegistry.getInstance().registerThreadLocalAccessor(threadLocalAccessor);
        };
    }
}
```

```

        // Enable automatic context propagation in Reactor 3.5.3+
        Hooks.enableAutomaticContextPropagation();
    };
}
}

```

[Объяснить с](#) 

Зависимости

Добавляем `io.projectreactor:reactor-core-micrometer`, чтобы воспользоваться классом `reactor.core.publisher.Hooks`.

```

<dependency>
  <groupId>io.projectreactor</groupId>
  <artifactId>reactor-core-micrometer</artifactId>
  <scope>provided</scope>
</dependency>

```

[Объяснить с](#) 

Результат

Реактивные приложения (в т.ч. Spring Cloud Gateway), участвующие в интеграционном потоке, поддерживают трассировку.

Сборка всех конфигураций

Вышеописанные конфигурации добавляем в `@Import` над главным классом `CustomTracingAutoConfiguration`. Автоконфигурация включается, когда свойство `management.tracing.enabled` имеет значение `true`.

```

import org.springframework.boot.actuate.autoconfigure.tracing.zipkin.ZipkinAutoConfiguration

@ConditionalOnProperty(name = "management.tracing.enabled", havingValue = "true")
@AutoConfiguration(before = ZipkinAutoConfiguration.class)
@Import({
    KafkaSenderConfiguration.class,
    ObservedAspectConfiguration.class,
    FeignObservedConfiguration.class,
    KafkaObservationEnablerConfig.class,
    SoapObservationConfiguration.class,
    ReactorObservationConfiguration.class
})
public class CustomTracingAutoConfiguration {
}

```

[Объяснить с](#) 

Использование

Подключение

Стартер подключается к приложению как зависимость в `pom.xml`.

```
<dependency>
  <groupId>ru.alfastrah.api</groupId>
  <artifactId>spring-boot-tracing-starter</artifactId>
  <version>1.1.0</version>
</dependency>
```

[Объяснить с 📄](#)

Затем в `application.yaml` добавляются свойства для его включения. Я упоминал их ранее, но размещу также здесь, чтобы были под рукой:

▸ [Конфигурация стартера и management в application.yaml](#)

Демонстрация результатов

Для воспроизведения полной картины появившихся возможностей соберем систему сервисов. Цель их создания - демонстрация трейса, оставленного при их взаимодействии друг с другом.

Каждый сервис оснащен стартером и демонстрирует его функционирование на примере перечисленных интеграций. Ни один из сервисов не имплементирует дополнительной логики работы с трейсами. Эту задачу полностью реализует стартер. Реализации/конфигурации тестовых приложений скрыты под спойлерами.

API-gateway

Принимает запросы на порту 8080 и маршрутизирует их на сервис **request-handler**, начинает трейс. Содержит конфигурацию единственного route.

▸ [Конфигурация route'a \(application.yaml\) для API-gateway](#)

Request-handler

Принимает запросы на порту 8081 от **api-gateway**, далее делает REST-запрос в сервис **producer-service** с помощью Feign-клиента.

▸ [Реализация Request-handler](#)

Consumer-service

Слушает сообщения в топике `demo-traces-topic`, реализует паттерн DLQ с помощью встроенных инструментов Spring:

- первая итерация обработки завершается ошибкой, помещает сообщение в топик `demo-traces-topic-retry`;
- вторая итерация успешна, отправляет запрос далее, в **soap-service**.

Демонстрирует обработку заголовков сообщения Kafka, продолжая ранее начатый трейс, а также добавление заголовка `traceparent` к SOAP-запросу.

Запуск и проверка

Для запуска демонстрации делаем POST запрос на **api-gateway**:

```
POST /trace-demo/start HTTP/1.1
Host: localhost:8080
Authorization: Bearer bearer.token.content
Content-Type: application/json
Content-Length: 34

{
  "message": "Hello, World!"
}
```

[Объяснить с](#) 

Посмотрим значимые логи сервисов. В них видим, что каждый сервис поучаствовал в обработке. В логах каждого сервиса присутствует `traceld 69bd61a67d70e718ad939eda62b5ba02`.

```
2026-03-20T18:03:02.925+03:00 INFO 21738 --- [api-gateway] [ctor-http-nio-4] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:03.763+03:00 INFO 21738 --- [api-gateway] [ctor-http-nio-4] [69bd61a67d70e718ad939eda62b5ba02]

2026-03-20T18:03:03.087+03:00 INFO 13874 --- [request-handler] [nio-8081-exec-1] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:03.118+03:00 DEBUG 13874 --- [request-handler] [nio-8081-exec-1] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:03.730+03:00 DEBUG 13874 --- [request-handler] [nio-8081-exec-1] [69bd61a67d70e718ad939eda62b5ba02]

2026-03-20T18:03:03.235+03:00 INFO 20771 --- [producer-service] [nio-8082-exec-9] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:03.241+03:00 INFO 20771 --- [producer-service] [nio-8082-exec-9] [69bd61a67d70e718ad939eda62b5ba02]

2026-03-20T18:03:03.647+03:00 INFO 16539 --- [consumer-service] [ntainer#0-0-C-1] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:03.665+03:00 ERROR 16539 --- [consumer-service] [ntainer#0-0-C-1] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:04.688+03:00 INFO 16539 --- [consumer-service] [r#0-retry-0-C-1] [69bd61a67d70e718ad939eda62b5ba02]
2026-03-20T18:03:04.704+03:00 INFO 16539 --- [consumer-service] [r#0-retry-0-C-1] [69bd61a67d70e718ad939eda62b5ba02]
    Address: http://localhost:8084/ws
    HttpMethod: POST
    Content-Type: text/xml
    ExchangeId: 0d943c99-2f23-457f-b102-8380c0ac6fab
    ServiceName: AnyPortService
    PortName: AnyPortPort
    PortTypeName: AnyPort
    Headers: {SOAPAction="", Accept=*/, traceparent=00-69bd61a67d70e718ad939eda62b5ba02-0000000000000000}
    Payload: <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"><soap:Body></soap:Body></soap:Envelope>

2026-03-20T18:03:04.832+03:00 INFO 16539 --- [consumer-service] [r#0-retry-0-C-1] [69bd61a67d70e718ad939eda62b5ba02]
    Address: http://localhost:8084/ws
    Content-Type: text/xml;charset=utf-8
    ResponseCode: 200
    ExchangeId: 0d943c99-2f23-457f-b102-8380c0ac6fab
    ServiceName: AnyPortService
    PortName: AnyPortPort
    PortTypeName: AnyPort
```

```
Headers: {date=Fri, 20 Mar 2026 15:03:04 GMT, SOAPAction="", Accept=text/xml, text/
Payload: <SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelop

2026-03-20T18:03:04.805+03:00 INFO 16568 --- [soap-service] [nio-8084-exec-6] [69bd61a
Объяснить с
```

Далее посмотрим, как трейс `69bd61a67d70e718ad939eda62b5ba02` выглядит в Zipkin. Если он развернут на localhost, то прямая ссылка на него выглядит так <http://localhost:9411/zipkin/traces/69bd61a67d70e718ad939eda62b5ba02>.

Таблица span’ов

Таблицу можно увидеть, нажав на кнопку “Span table” в интерфейсе трейса. В таблице можно найти имена сервисов, участвующих во взаимодействии и дополнительную информацию. Каждый сервис присутствует, значит все сконфигурировано корректно.

Span ID	Service name	Span name	Start time	Duration
ad939eda62b5ba02	api-gateway	http post	03/20 18:03:02.695	1.068s
37863a0356495ac0	api-gateway	security filterchain before	03/20 18:03:02.696	215.113ms
bc7251525b5041c9	api-gateway	authorize exchange	03/20 18:03:02.909	1.819ms
05d2ca72fd9e68f7	api-gateway	secured request	03/20 18:03:02.911	850.960ms
edb8315a0477fd9a	api-gateway	http post	03/20 18:03:02.932	825.168ms
3dd0d6ee2d003442	request-handler	http post /trace-demo/start	03/20 18:03:03.006	744.776ms
9afd619825b507ab	request-handler	http post	03/20 18:03:03.127	602.950ms
0996a1356bf5ebd7	producer-service	http post /trace-demo/pr...	03/20 18:03:03.202	524.373ms
33409127483821b1	producer-service	demo-traces-topic send	03/20 18:03:03.243	463.835ms
05d5ea464727cedd	consumer-service	demo-traces-topic receive	03/20 18:03:03.594	500.772ms
fd2a270c73531479	consumer-service	demo-traces-topic-retry ...	03/20 18:03:04.071	23.922ms
328fa4ca28e3c688	consumer-service	demo-traces-topic-retry ...	03/20 18:03:04.094	3.391ms
37ea00b17e36fb56	consumer-service	demo-traces-topic-retry ...	03/20 18:03:04.688	146.559ms
950931dbcb208d37	soap-service	http post	03/20 18:03:04.762	60.392ms
dddfee194556b0c8	api-gateway	security filterchain after	03/20 18:03:03.762	300.000µs

span table

Таймлайн

На таймлайне видны разрывы. Они соответствуют времени, которое ушло на то, чтобы сервис увидел, что в топике появилось сообщение и на ожидание перед повторной обработкой (retry).

timeline

Дерево span’ов

Дерево показывает подробности обработки.

span tree

Заключение

Путем добавления нескольких компонентов в конфигурацию, полученную как результат [первой части статьи](#), мы все-таки достигли желаемой цели: теперь трейс содержит всю цепочку взаимодействий сервисов, которые участвовали в итерации обработки запроса.

Надеюсь, Вам было интересно.

Исходный код стартера можно [найти по ссылке](#).

Благодарности

Большое спасибо моему доброму другу и коллеге Андрею за вклад в развитие описанной функциональности. Вместе мы довели ее до состояния, когда она помогает не только нашей команде, но и коллегам из смежных команд, которые ее используют.

Теги: [w3c](#), [starter](#), [tracing](#), [zipkin](#), [spring-boot](#), [трейсы](#)

Хабы: [Блог компании АльфаСтрахование](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



16K+

Охват за 30 дней

АльфаСтрахование

[Сайт](#)



4K+

4

Охват за 30 дней

Карма

Носов Никита [@nosbuka](#)

Пользователь

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

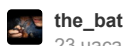
Перейти

[Перейти в поток Бэкенд](#)

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+97

31

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

27K

Обзор

Перевод

+72

45

85



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин

9.4K

Кейс

+52

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой

7 мин

12K

Обзор

+37

59

18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний

14 мин

10K

Мнение

+28

33

11



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

🕒 5 мин 👁 8.8K

Обзор

📦 +27

🔖 10

💬 0



mariklolik

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

🔗 Средний 🕒 7 мин 👁 7.5K

Кейс

📦 +23

🔖 23

💬 6



nick_dyuba

23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин 👁 7.1K

Recovery Mode

📦 +22

🔖 8

💬 5



Mimizavr

вчера в 12:21

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин 👁 7K

📦 +17

🔖 9

💬 7



infgotoinf

11 часов назад

Perl — зря забытый язык программирования?

🕒 11 мин 👁 8.6K

Из песочницы

📦 +16

🔖 13

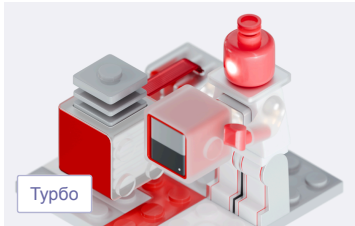
💬 10

Тысяча нюансов: как управлять производством лекарств

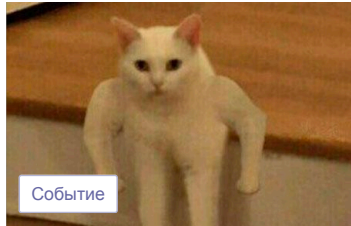
Турбо

Показать еще

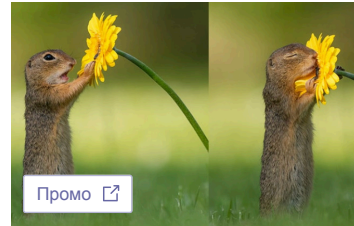
МИНУТОЧКУ ВНИМАНИЯ



Собери облачный пазл и выиграй призы



Качаем к лету хард- и софт-скиллы на айтишных ивентах



Запахло весной и скидками в Промокодусе

ИНФОРМАЦИЯ

Сайт	alfastrah.ru
Дата регистрации	25 апреля 2019
Дата основания	5 февраля 1992
Численность	5 001–10 000 человек
Местоположение	Россия

БЛОГ НА ХАБРЕ

19 часов назад

Трейсы в Spring Boot 3 с использованием Zipkin и Kafka в качестве транспорта. Часть 2

👁 5.3K 💬 1

16 апр в 20:08

Никто не любит писать тесты, но ИИ может исправить это

9.1K 13

3 апр в 17:50

SIP- и Push-уведомления. Опыт с Android-устройствами

4.6K 0

26 мар в 13:16

Calico на Orange Pi не поднимается

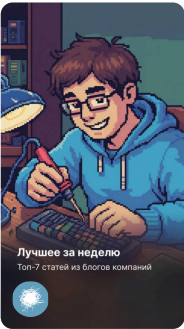
5.4K 2

16 мар в 17:32

От простыней к DSL: эволюция UI-тестов на Java

6.5K 2

ИСТОРИИ



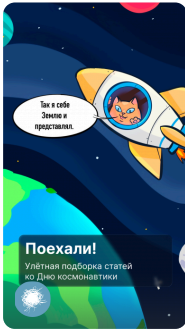
Годнота из блогов компаний



Только 10% решат эту головоломку



Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости

Информация

Устройство сайта
Для авторов

Услуги

Корпоративный блог
Медийная реклама

Хабы

Компании

Авторы

Песочница

Для компаний

Документы

Соглашение

Конфиденциальность

Нативные проекты

Образовательные программы

Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr